ShARC: Codebase Technical Review

Shift-robust Arc Routing under Capacity Constraints

Agriya Yadav

April 2026

Contents

Project Overview

ShARC (also referred to as ReMAR in experiments) addresses the *Hierarchical Capacitated Arc Routing Problem* (H-CARP) under distribution shifts. The core research question is: can a deep reinforcement learning (RL) policy trained with a risk-averse CVaR objective outperform classical heuristics and risk-neutral RL policies when demand, cost, or task-availability shift at test time?

The repository contains four co-equal components:

1. Classical heuristic and exact solvers as baselines (ILS, EA, ACO, LP).
2. A batched, vectorised RL environment modelling H-CARP with shift injection.
3. An encoder–decoder neural policy with attention and shift conditioning.
4. A training harness implementing both REINFORCE and CVaR-REINFORCE.

Problem Formulation

H-CARP definition

Given an undirected graph $G = (V, E)$ with depot v_0, a fleet of M vehicles each with capacity C, and a set of *required* arcs $R \subseteq E$ partitioned into priority classes $P = \{1, 2, 3\}$, the objective is to find routes $\tau_1, \ldots, \tau_M$ such that every arc $e \in R$ is serviced exactly once and the **lexicographic makespan vector** $\mathbf{T} = (T_1, T_2, T_3)$ is minimised:

$$T_k = \max_{m=1}^{M} \text{ completion time of vehicle } m \text{ after servicing all class-}k \text{ arcs.} \tag{1}$$

Lexicographic minimisation means T_1 has strict priority over T_2, and T_2 over T_3. The scalar proxy used in training is $-(10^4 T_1 + 10^2 T_2 + T_3)$.

Distribution shift model

At evaluation time the environment applies a *shift* parameterised by severity $s \in [0, 1]$:

$$\begin{gathered} d_e \leftarrow d_e\,(1 + s\,\delta_d), \quad c_{ij} \leftarrow c_{ij}\,(1 + s\,\delta_c), \\ \text{arc } e \text{ unavailable with probability } s\,(1 - p_{\text{avail}}). \end{gathered} \tag{2}$$

The shift context vector $(s\,\delta_d,\ s\,\delta_c,\ p_{\text{avail}})$ is passed to the policy decoder, enabling explicit conditioning.

Repository Structure

Directory / File	Role
`baseline/meta.py`	Base class + EA, ACO, cheapest-insertion heuristics
`baseline/lp.py`	Gurobi MILP formulation (LP-HCARP)
`common/ops.py`	Graph parsing, Floyd–Warshall, instance loading
`common/cal_reward.py`	Lexicographic reward (Numba JIT)
`common/intra.py`, `inter.py`	2-opt local search (Numba JIT)
`models/encoder.py`	Transformer arc encoder
`models/decoder.py`	Attention pointer decoder
`models/policy.py`	Full encoder–decoder policy
`env/hcarp_env.py`	Batched NumPy RL environment
`env/shift.py`	Distribution shift scheduler
`training/train.py`	REINFORCE & CVaR-REINFORCE training loop
`evaluation/`	Metric computation & severity sweeps
`data/`	Instance generators and datasets
`results/`	Experimental outputs (CSV, PNG)

The codebase totals approximately 2,400 lines of Python, with the environment, training loop, and baseline module each contributing ∼300–370 lines.

Data Format

Instances are stored as compressed NumPy archives (`.npz`) with the following schema:

Key	Shape	dtype	Meaning
`req`	$(n_r, 6)$	float32	Required arcs: (u, v, d, p, s, t)
`nonreq`	$(n_t, 6)$	float32	Traversal arcs (same columns, $d = p = s = 0$)
`C`	scalar	float32	Vehicle capacity
`M`	scalar	int32	Fleet size
`P`	scalar	int32	Number of priority classes (always 3)

Columns: u, v = node indices (depot = 0); d = demand; $p \in \{1, 2, 3\}$ = priority; s = service time; t = travel time. Datasets follow the path convention `data/{split}/{size}/{id}.npz`.

Baseline Solvers

All baselines inherit from `BaseHCARP` (`baseline/meta.py`), which provides instance parsing, feasibility checking, and objective evaluation.

Iterative Local Search (ILS)

Generates 20 random cheapest-insertion tours, applies three rounds of intra- and inter-route 2-opt refinement via `common/local_search.py`, and returns the best solution. Complexity per call: $O(k \cdot n^2)$ where $k = 3$ local-search rounds.

Evolutionary Algorithm (EA)

A steady-state genetic algorithm with population 200, tournament size 4, mutation rate 0.5, and crossover rate 0.9. Crossover is segment-based with capacity re-checking; mutation applies a

single 2-opt move. Two elites are preserved per generation. Runs for 50 epochs by default.

Ant Colony Optimisation (ACO)

50 ants construct probabilistic tours using pheromone matrix τ and heuristic η with exponents $\alpha = \beta = 1.0$. Pheromone evaporation rate $\rho = 0.5$. Optional local search after construction. Default 500 epochs.

LP-HCARP (Gurobi)

A mixed-integer linear programme solved by Gurobi with a 600-second wall-clock limit. Decision variables:

- $x_{m,a} \in \{0, 1\}$: vehicle m services arc a.
- $y_{m,a,k} \in \mathbb{Z}_{\geq 0}$: deadhead traversal count.
- $t_{m,k} \in \mathbb{R}_{\geq 0}$: completion time for vehicle m at priority class k.

Subtour elimination is enforced via lazy constraints using NetworkX strongly connected component detection. The weighted-sum objective $10^3 T_1 + 10\, T_2 + 0.1\, T_3$ approximates lexicographic minimisation. Returns `None` on timeout.

Neural Policy

Architecture

`HCARPPolicy` (`models/policy.py`) is a Transformer encoder–pointer decoder:

Encoder (`ArcEncoder`). Three layers of pre-LN multi-head self-attention ($d_{\text{model}} = 128$, 8 heads, $d_{\text{ff}} = 512$). Input features per arc: service time, demand, learned priority class embedding ($d_{\text{clss}} = 16$), and a depot indicator flag. Output: contextual arc embeddings $\mathbf{H} \in \mathbb{R}^{B \times (n+1) \times d_{\text{model}}}$.

Decoder (`AttentionDecoder`). A single-step pointer network. At each decision step the context vector is

$$\mathbf{c} = \big[\mathbf{h}_{\text{cur}} \parallel \text{rem\_cap} \parallel \text{veh\_time} \parallel \text{budget} \parallel \mathbf{e}_{\text{shift}}\big], \tag{3}$$

where $\mathbf{e}_{\text{shift}} \in \mathbb{R}^{d_{\text{shift}}=8}$ is a learned embedding of the shift context. Compatibility logits are $u_i = \text{clip}\Big(\mathbf{q}^\top \mathbf{k}_i / \sqrt{d},\ -\text{clip},\ \text{clip}\Big)$ with clip $= 10$. Infeasible actions receive $-\infty$ before `log_softmax`.

Policy interface

```
policy.encode(obs)            # -> arc_emb [B, n+1, d_model]
policy.act(obs, arc_emb)      # -> actions [B], log_probs [B]
policy.rollout(env)           # -> actions [B,T], log_probs [B,T],
    rewards [B]
```

Greedy decoding (argmax) is used at evaluation; categorical sampling during training.

RL Environment

`HCARPEnv` (`env/hcarp_env.py`) runs B instances in parallel using pure NumPy (no GPU dependency for simulation). The action space is $\{0, 1, \ldots, n\}$ where 0 terminates the current vehicle's route and advances to the next. Observations include:

Tensor	Shape	Type
`adj`	$[B, n{+}1, n{+}1]$	static
`service_time`	$[B, n{+}1]$	static
`demand`	$[B, n{+}1]$	static
`clss`	$[B, n{+}1]$	static
`visited`	$[B, n{+}1]$	dynamic
`cur_arc`	$[B, M]$	dynamic
`remaining_cap`	$[B, M]$	dynamic
`vehicle_time`	$[B, M]$	dynamic
`budget`	$[B]$	dynamic
`shift_context`	$[B, 3]$	context
`action_mask`	$[B, n{+}1]$	feasibility

The reward is zero at all intermediate steps; at termination: $r = -(10^4 T_1 + 10^2 T_2 + T_3)$.

Shift Scheduler

`ShiftScheduler` (`env/shift.py`) supports three modes:

`curriculum`: Perturbation magnitude grows linearly over `warmup_steps` (default 1000) from 0 to maximum ($\pm 30\%$ demand/cost, 30% task removal).
`uniform`: Samples $\delta_d, \delta_c, p_{\text{avail}}$ uniformly from fixed ranges.
`adversarial`: Maximises expected loss (intended for future adversarial training).

Training Algorithms

Both algorithms share the same environment rollout and differ only in how the policy gradient is weighted. Let R_i be the stochastic rollout return for instance i and $\hat{R}_i$ the greedy (baseline) return.

Standard REINFORCE

$$\mathcal{L}_{\text{RN}} = -\frac{1}{B}\sum_{i=1}^{B} \widetilde{A}_i \cdot \log \pi_\theta(\tau_i), \qquad \widetilde{A}_i = \frac{(R_i - \hat{R}_i) - \mu_A}{\sigma_A}. \tag{4}$$

CVaR-REINFORCE

Only the worst α-fraction ($\alpha = 0.1$) of trajectories receives gradient signal:

$$\text{VaR}_\alpha = \text{quantile}_\alpha\{R_i\}, \tag{5}$$

$$w_i = \frac{\mathbf{1}[R_i \le \text{VaR}_\alpha]}{\alpha B}, \tag{6}$$

$$\mathcal{L}_{\text{CVaR}} = -\sum_{i=1}^{B} w_i \left(R_i - \widehat{\text{CVaR}}_\alpha\right) \cdot \log \pi_\theta(\tau_i), \tag{7}$$

where $\widehat{\text{CVaR}}_\alpha$ is the CVaR of the greedy baseline (not the mean). This concentrates updates on adverse outcomes, training the policy to improve its worst-case performance.

Training configuration

Hyperparameter	Value
Batch size	32 instances
Epochs	200
Optimizer	Adam, lr $= 10^{-4}$
Gradient clip	1.0
Validation interval	10 epochs
d_{model}	128
Attention heads	8
Encoder layers	3
Feed-forward dim	512
α (CVaR)	0.1

Local Search

All heuristics and post-processing apply 2-opt moves compiled with Numba JIT for speed.

Intra-route (`common/intra.py`): Tries all arc-pair reversals within a single vehicle's route; accepts if $\Delta\text{cost} < -\varepsilon$.

Inter-route (`common/inter.py`): For each pair of vehicles and each priority class k, tries swapping one arc from each vehicle; accepts if cost decreases and capacity remains feasible. Iterates up to EPS $= 1000$ rounds.

Two *variants* are supported:

- **P (Priority):** Optimises each tier sequentially; higher tiers are not degraded.
- **U (Uniform):** Optimises the lexicographic scalar directly.

Evaluation

Metrics

$$\text{Mean} = \frac{1}{N}\sum_i R_i, \tag{8}$$

$$\text{CVaR}_\alpha = \mathbb{E}[R \mid R \leq \text{VaR}_\alpha], \tag{9}$$

$$\text{Gap} = \frac{\text{Baseline obj} - \text{Policy obj}}{\text{Baseline obj}} \times 100\%. \tag{10}$$

Severity sweep

`ShiftEvaluator.sweep` evaluates each policy at eleven severity levels $s \in \{0.0, 0.1, \ldots, 1.0\}$ using greedy decoding, producing the CSV `results/severity_sweep_v2.csv`. The sweep is the primary benchmark: it characterises graceful degradation under increasing distribution shift.

Summary of results

Method	Mean reward (s=0)	$\text{CVaR}_{0.1}$ (s=1)
ILS	competitive	degrades significantly
EA	competitive	degrades significantly
ACO	competitive	degrades significantly
LP-HCARP	optimal (slow)	N/A (timeout-prone)
Risk-neutral RL	near-optimal	degrades ∼20% more than CVaR
CVaR-RL	near-optimal	best tail robustness

At $s = 0$ all learned policies match baselines. At $s = 1$ CVaR-REINFORCE yields ∼20% better CVaR than risk-neutral RL, confirming that tail-risk optimisation transfers to distribution-shift robustness.

Design Decisions and Trade-offs

Decision	Choice	Rationale
Objective function	Lexicographic $\mathbf{T} = (T_1, T_2, T_3)$	Hard priority ordering; scalar proxies lose the lexicographic guarantee
Reward shaping	$-10^4 T_1 - 10^2 T_2 - T_3$	Approximates lexicographic order in a single scalar for gradient flow
Encoder	Transformer self-attention	Captures global arc interactions; naturally handles variable-length instances
Decoder	Pointer network with masking	Encodes feasibility directly; avoids invalid action sampling
Shift conditioning	Explicit shift-context embedding in decoder	Lets policy adapt without distribution mismatch at test time
Risk objective	CVaR-REINFORCE ($\alpha = 0.1$)	Concentrates gradient on worst 10% outcomes; improves tail without hurting mean
Parallelism	Numba JIT for local search	Python-speed reward evaluation was the bottleneck; JIT gives 10×+ speedup
Baseline comparison	Greedy rollout (same policy, no sampling)	Isolates routing quality from stochastic variance

Dependencies

Package	Role
`numpy`	Dense array operations throughout
`torch`	Neural network, Adam, categorical distributions
`numba`	JIT compilation for reward and local search loops
`networkx`	Shortest paths, SCC detection for LP subtour elimination
`gurobipy`	Commercial MILP solver (LP-HCARP baseline)
`osmnx`	OpenStreetMap graph extraction for instance generation
`pandas`	Results aggregation and CSV I/O
`matplotlib`	Gantt charts, severity plots

Identified Issues and Recommendations

1. **Gurobi dependency.** The LP baseline requires a commercial licence. Replacing it with an open-source MILP solver (e.g. HiGHS via `scipy.optimize.milp`) would lower the barrier for reproducibility.
2. **Numba cold-start overhead.** JIT compilation adds ∼10–30 seconds on first call. Ahead-of-time caching (`@nb.njit(cache=True)`) should be enabled uniformly in `intra.py`, `inter.py`, and `cal_reward.py`.
3. **LP timeout with no fallback.** `LPHCARP` returns `None` on timeout; callers in `run_all_baselines.py` should handle this gracefully (e.g. fall back to ILS result).
4. **Single-seed evaluation.** The severity sweep results in `severity_sweep_v2.csv` appear to come from a single random seed. Reporting means and standard deviations across at least 5 seeds would strengthen statistical claims.
5. **CVaR weight degeneracy.** When $\alpha B < 1$ (small batches), the CVaR mask may select zero instances. A guard ensuring at least one instance is selected ($\max(1, \lfloor \alpha B \rfloor)$) would prevent silent zero-gradient steps.
6. **Adversarial shift mode stub.** `ShiftScheduler` exposes an `"adversarial"` mode but it is not implemented. Either implement it or remove the option to avoid silent no-ops.
7. **Missing `requirements.txt` pinning.** The file lists package names without version bounds. Pinning (e.g. with `pip-compile`) would make environment reproduction reliable.

Summary

ShARC is a well-structured, research-grade codebase of ∼2,400 lines that cleanly separates concerns across environment, model, training, evaluation, and baselines. The core algorithmic contribution—CVaR-REINFORCE with explicit shift conditioning—is correctly implemented and shows measurable tail-robustness gains at high severity. The Numba-accelerated local search is a practical engineering strength. The main weaknesses are a hard Gurobi dependency, single-seed evaluation, and a handful of unguarded edge cases in the CVaR weighting and LP fallback logic. Addressing these would bring the codebase to publication-ready quality.